

# Security and Access Control

Internal identity, trust, secrets and privileged-control model.



## Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

## Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository:  
rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @  
3225f7fa531a6b07185fb5e034f7930f8bd8b571

# Summary and how to use this document

Internal identity, trust, secrets and privileged-control model. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

## Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

## Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

## Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

## Needs source confirmation

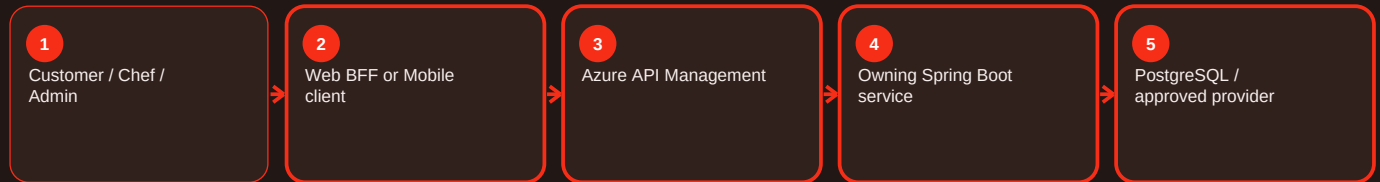
A referenced artifact or exact value could not be verified and is therefore not invented.

## Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

# How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



## Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

# Security layers - Overview and responsibility

## Current implementation

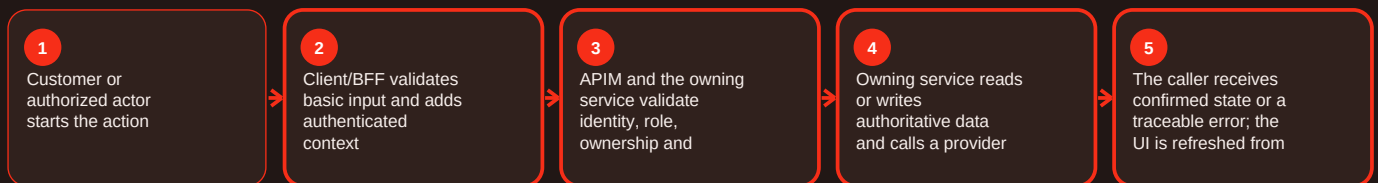
### Summary

Security layers is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For security layers, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Security layers - Functional behavior and data

## Current implementation

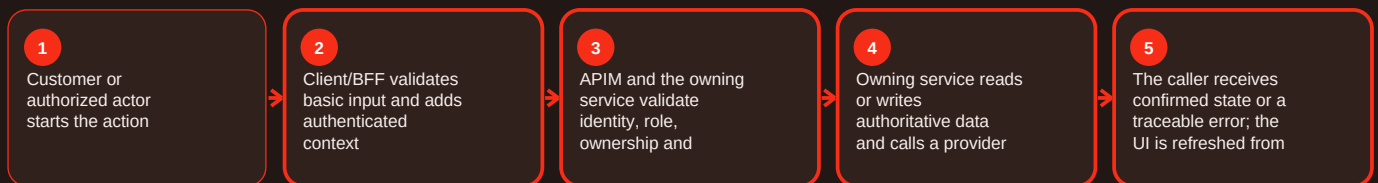
### Summary

Security layers is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For security layers, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Security layers - Operations, errors and deployment

## Current implementation

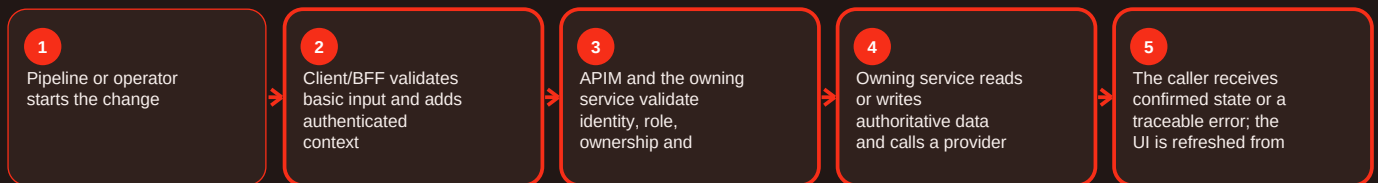
### Summary

Security layers is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For security layers, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Threat boundaries - Overview and responsibility

## Current implementation

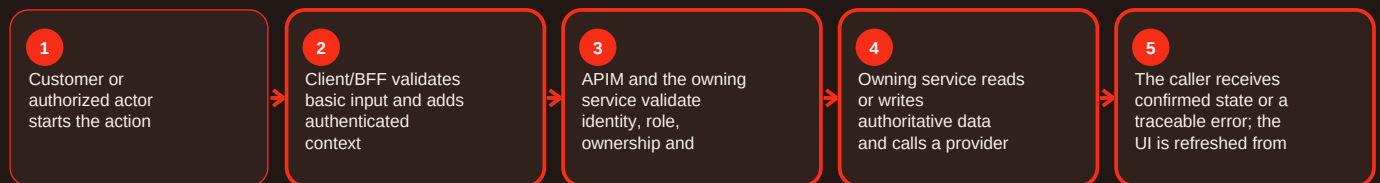
### Summary

Threat boundaries is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For threat boundaries, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Threat boundaries - Functional behavior and data

## Current implementation

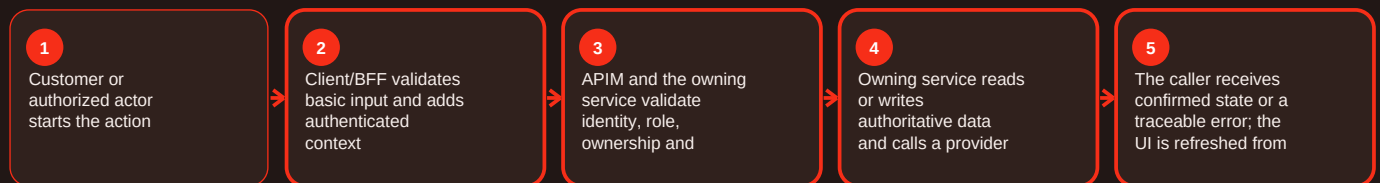
### Summary

Threat boundaries is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For threat boundaries, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.



# Threat boundaries - Operations, errors and deployment

## Current implementation

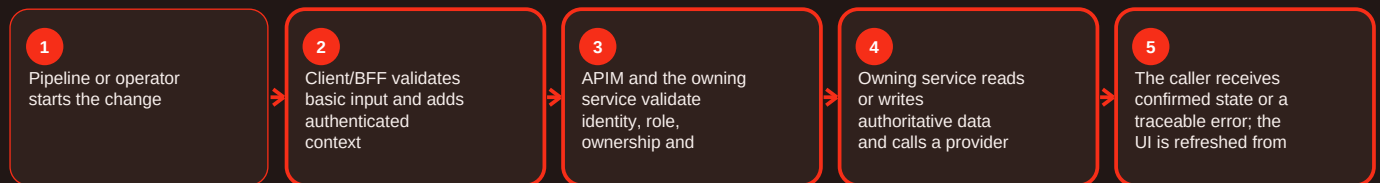
### Summary

Threat boundaries is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For threat boundaries, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Otp identity - Overview and responsibility

## Current implementation

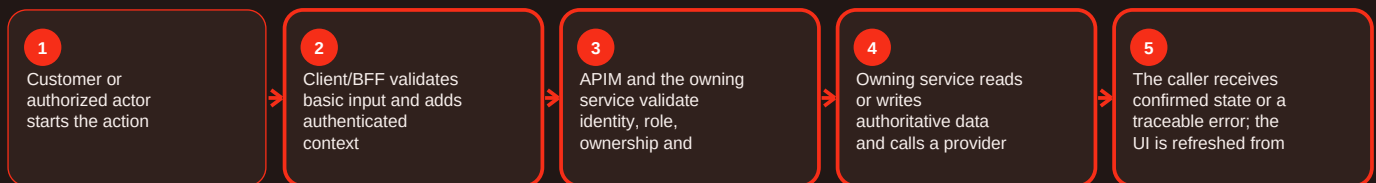
### Summary

Otp identity is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For OTP identity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Otp identity - Functional behavior and data

## Current implementation

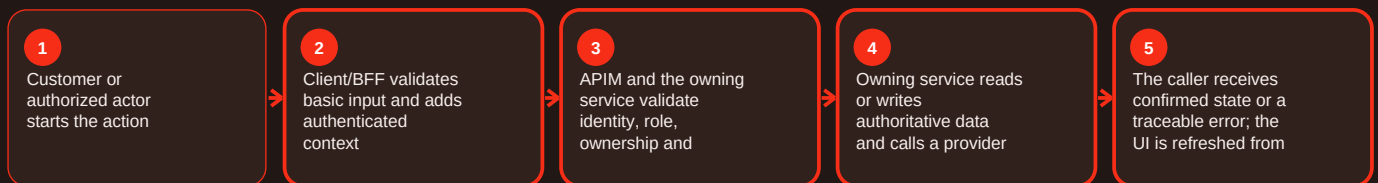
### Summary

Otp identity is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For OTP identity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Otp identity - Operations, errors and deployment

## Current implementation

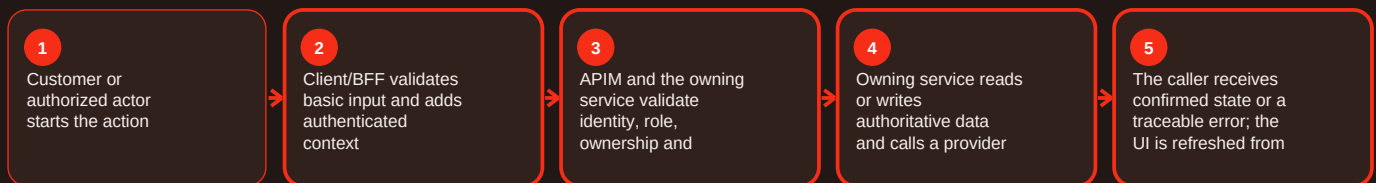
### Summary

Otp identity is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For OTP identity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Jwt validation - Overview and responsibility

## Current implementation

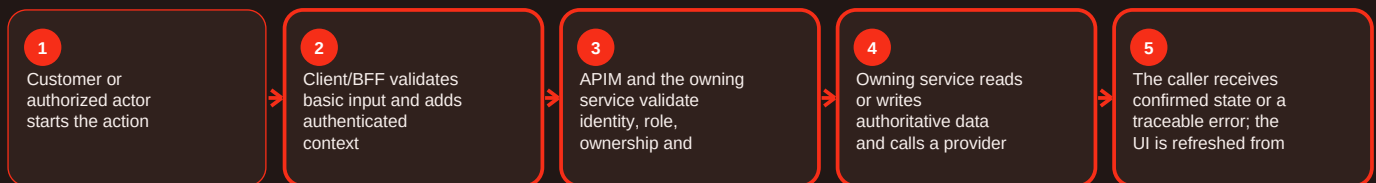
### Summary

Jwt validation is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For JWT validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Jwt validation - Functional behavior and data

## Current implementation

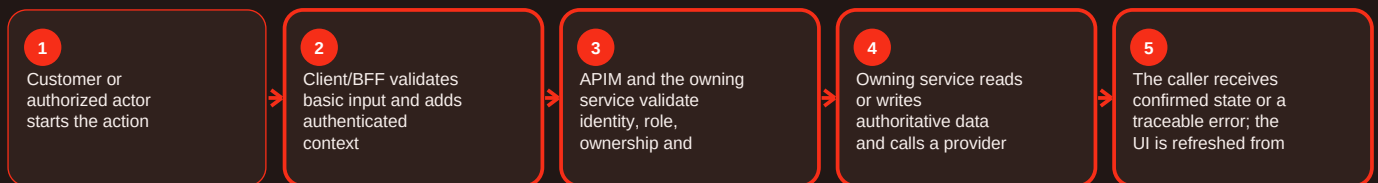
### Summary

Jwt validation is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For JWT validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Jwt validation - Operations, errors and deployment

## Current implementation

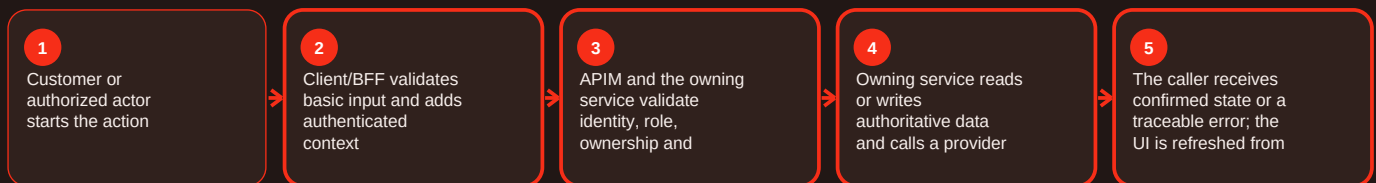
### Summary

Jwt validation is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For JWT validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Rbac - Overview and responsibility

## Current implementation

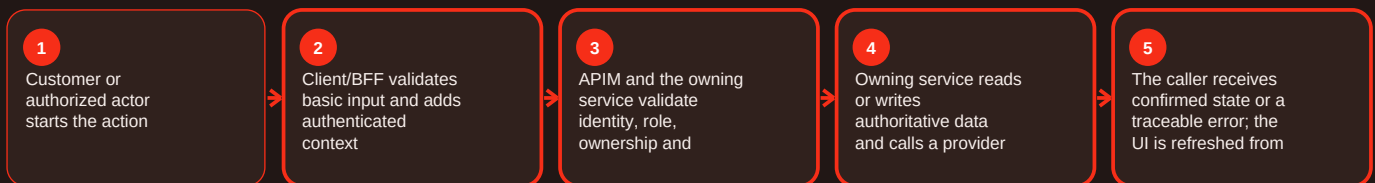
### Summary

Rbac is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For RBAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.



# Rbac - Functional behavior and data

## Current implementation

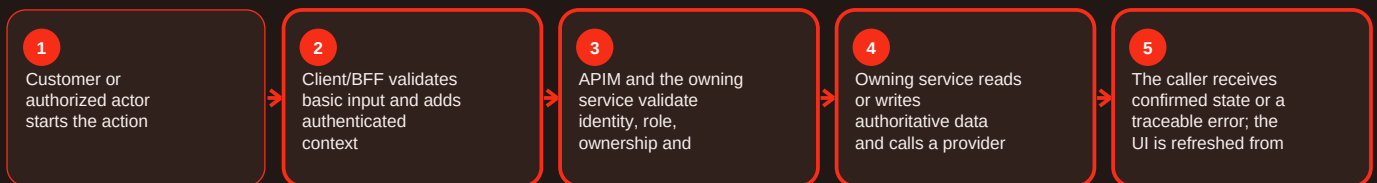
### Summary

Rbac is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For RBAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Rbac - Operations, errors and deployment

## Current implementation

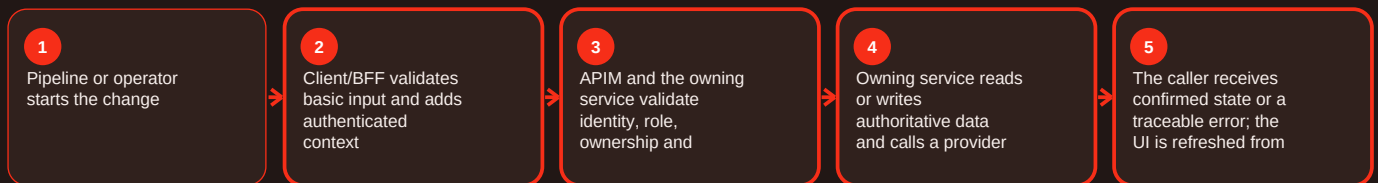
### Summary

Rbac is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For RBAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Ownership - Overview and responsibility

## Current implementation

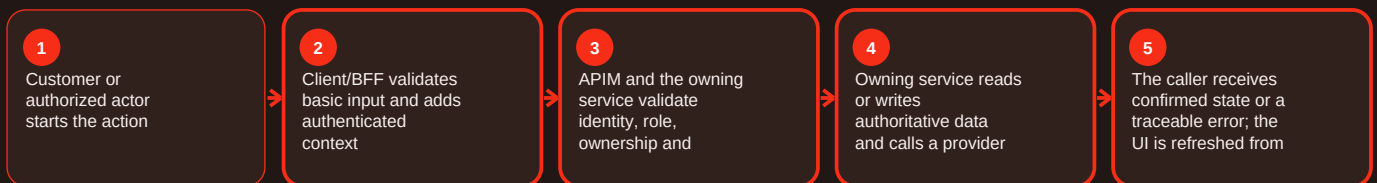
### Summary

Ownership is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For ownership, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Ownership - Functional behavior and data

## Current implementation

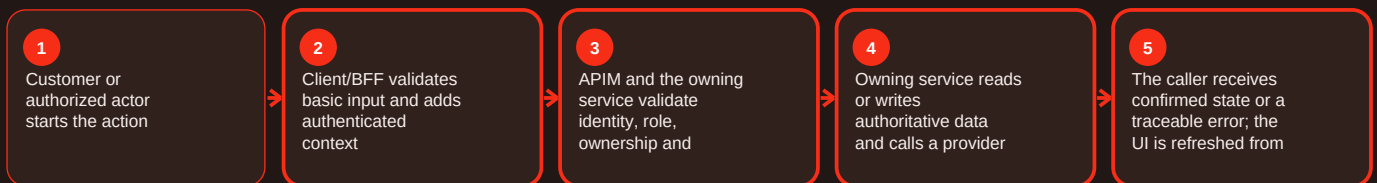
### Summary

Ownership is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For ownership, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Ownership - Operations, errors and deployment

## Current implementation

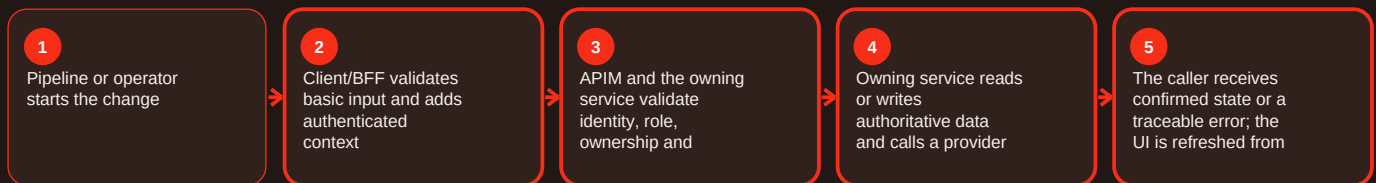
### Summary

Ownership is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For ownership, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Chef / admin isolation - Overview and responsibility

## Current implementation

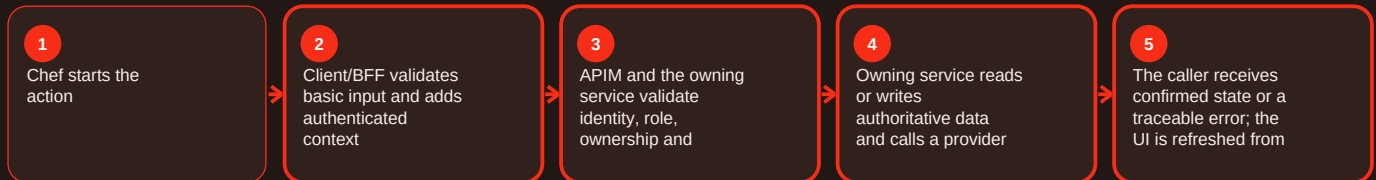
### Summary

Chef / admin isolation is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef/admin isolation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Chef / admin isolation - Functional behavior and data

## Current implementation

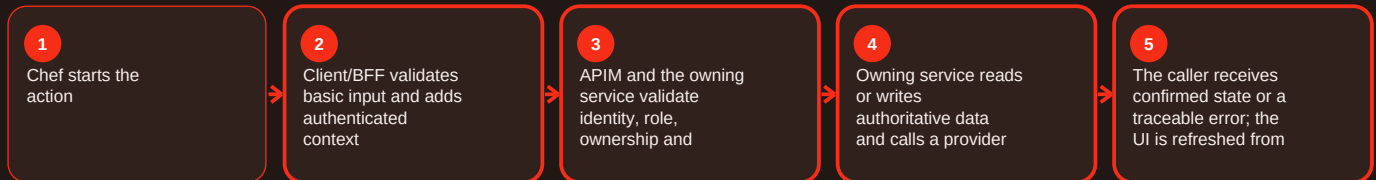
### Summary

Chef / admin isolation is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef/admin isolation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Chef / admin isolation - Operations, errors and deployment

## Current implementation

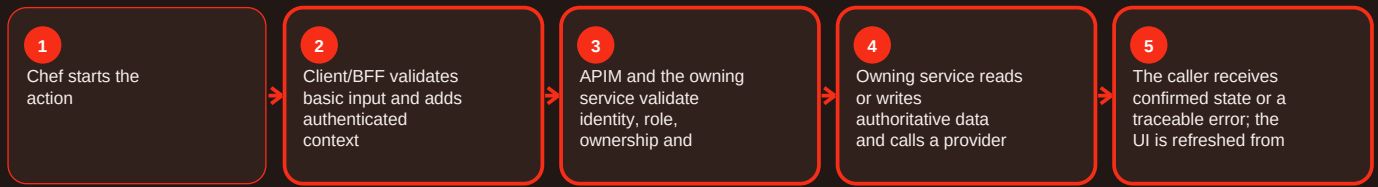
### Summary

Chef / admin isolation is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef/admin isolation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.



# Http-only cookies - Overview and responsibility

## Current implementation

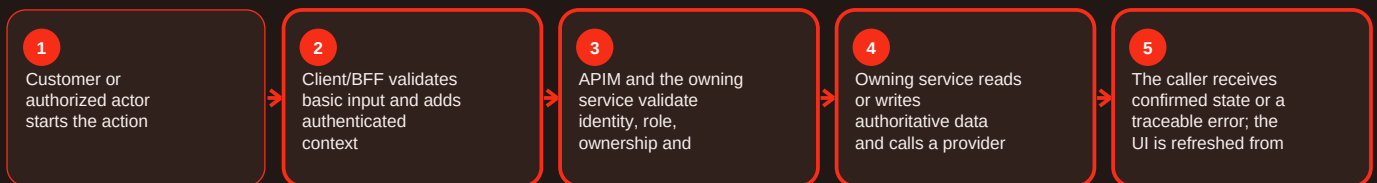
### Summary

Http-only cookies is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For HTTP-only cookies, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Http-only cookies - Functional behavior and data

## Current implementation

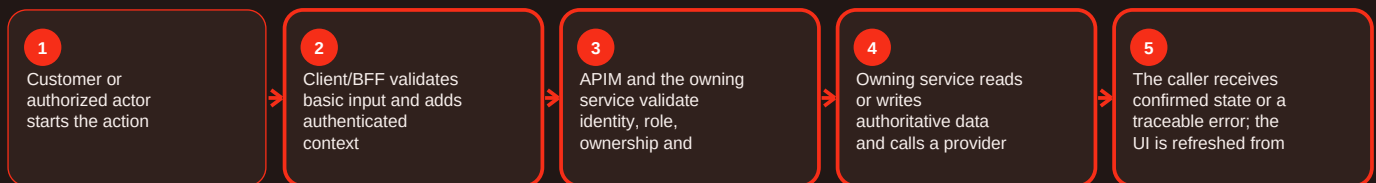
### Summary

Http-only cookies is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For HTTP-only cookies, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Http-only cookies - Operations, errors and deployment

## Current implementation

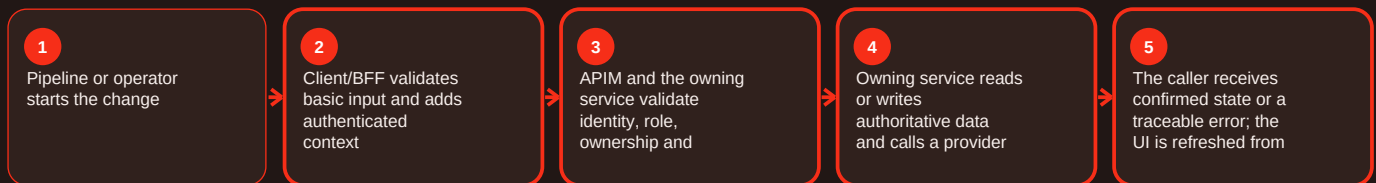
### Summary

Http-only cookies is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For HTTP-only cookies, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Mobile secure tokens - Overview and responsibility

Foundation / partial

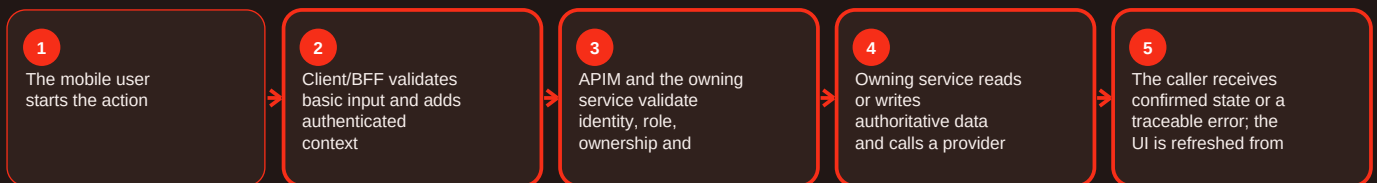
## Summary

Mobile secure tokens is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

## Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile secure tokens, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

## Process flow



## Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

## Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Mobile secure tokens - Functional behavior and data

Foundation / partial

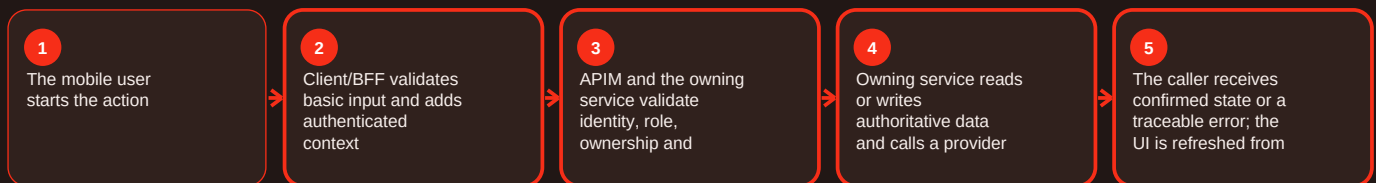
## Summary

Mobile secure tokens is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

## Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile secure tokens, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

## Process flow



## Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

## Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:  
 docs/handover/2026-07-30-customer-mobile-auth-foundation.md;  
 docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Mobile secure tokens - Operations, errors and deployment

Foundation / partial

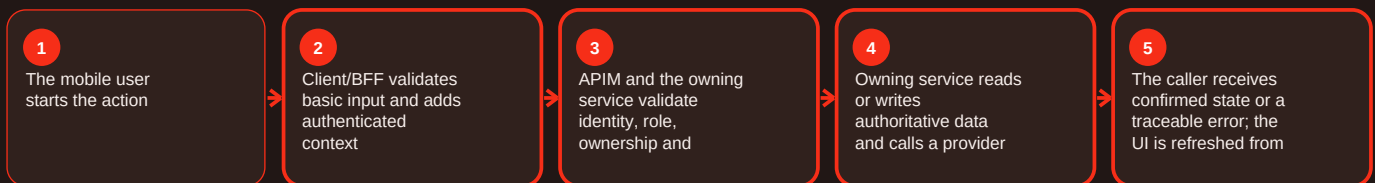
## Summary

Mobile secure tokens is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

## Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile secure tokens, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

## Process flow



## Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

## Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Pkce / mfa - Overview and responsibility

Foundation / partial

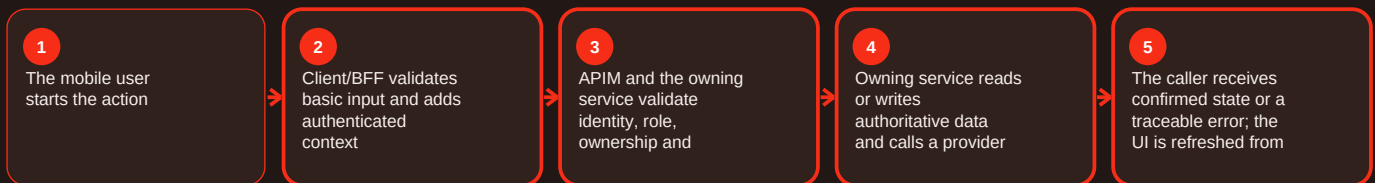
## Summary

Pkce / mfa is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

## Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For PKCE/MFA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

## Process flow



## Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

## Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:  
docs/handover/2026-07-30-customer-mobile-auth-foundation.md;  
docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Pkce / mfa - Functional behavior and data

Foundation / partial

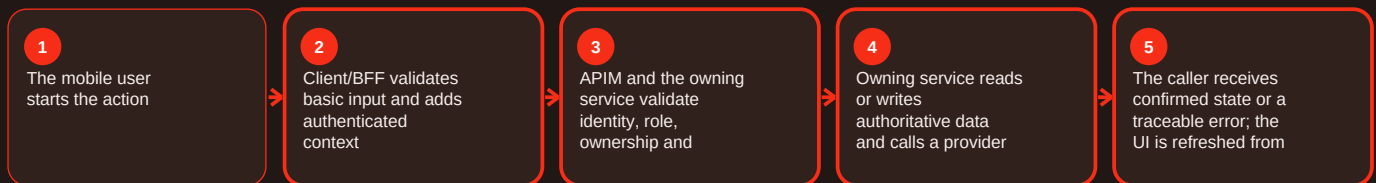
## Summary

Pkce / mfa is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

## Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For PKCE/MFA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

## Process flow



## Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

## Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence:  
docs/handover/2026-07-30-customer-mobile-auth-foundation.md;  
docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.



# Pkce / mfa - Operations, errors and deployment

Foundation / partial

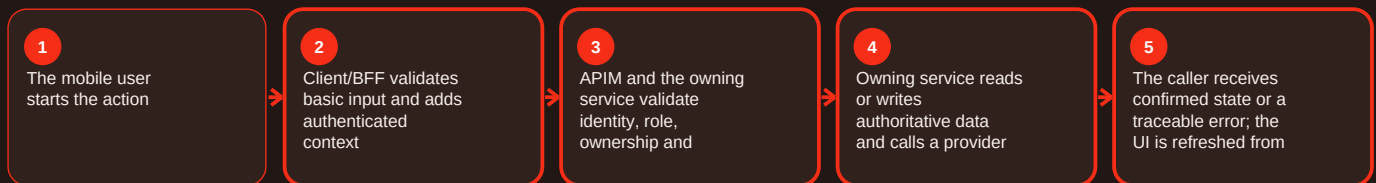
## Summary

Pkce / mfa is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

## Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For PKCE/MFA, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

## Process flow



## Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

## Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Secret storage - Overview and responsibility

## Current implementation

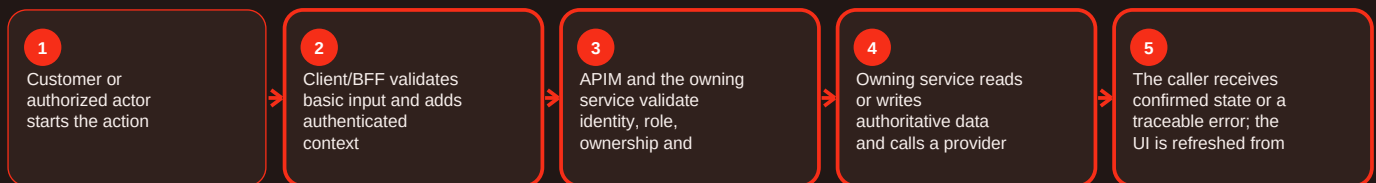
### Summary

Secret storage is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For secret storage, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Secret storage - Functional behavior and data

## Current implementation

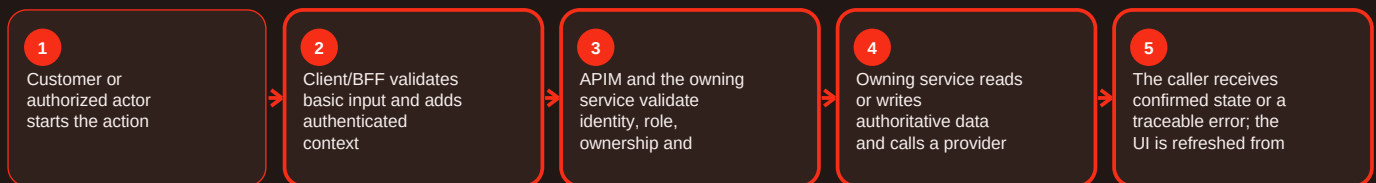
### Summary

Secret storage is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For secret storage, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Secret storage - Operations, errors and deployment

## Current implementation

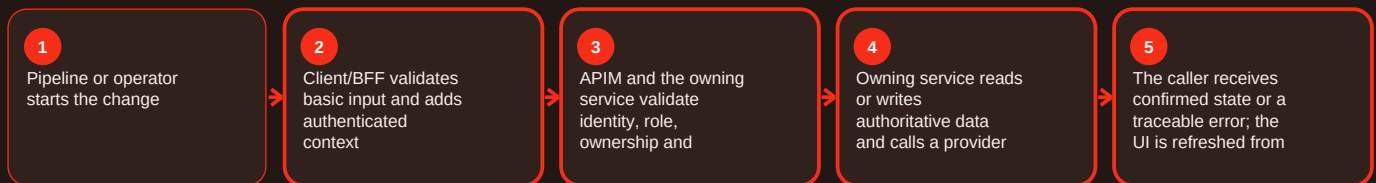
### Summary

Secret storage is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For secret storage, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Starter-secret prohibition - Overview and responsibility

## Current implementation

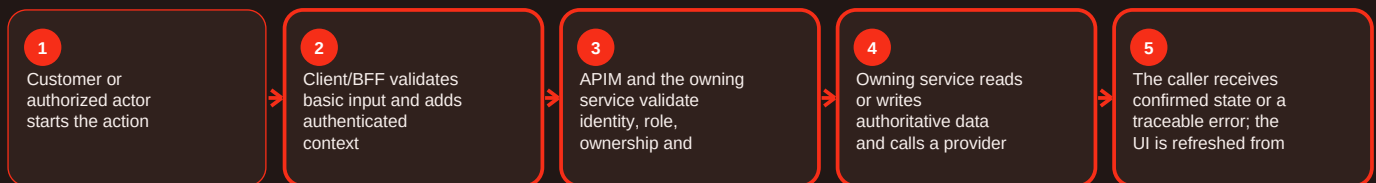
### Summary

Starter-secret prohibition is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For starter-secret prohibition, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Starter-secret prohibition - Functional behavior and data

## Current implementation

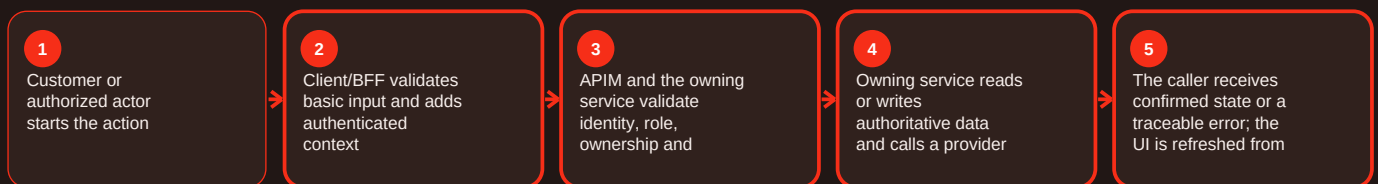
### Summary

Starter-secret prohibition is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For starter-secret prohibition, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Starter-secret prohibition - Operations, errors and deployment

## Current implementation

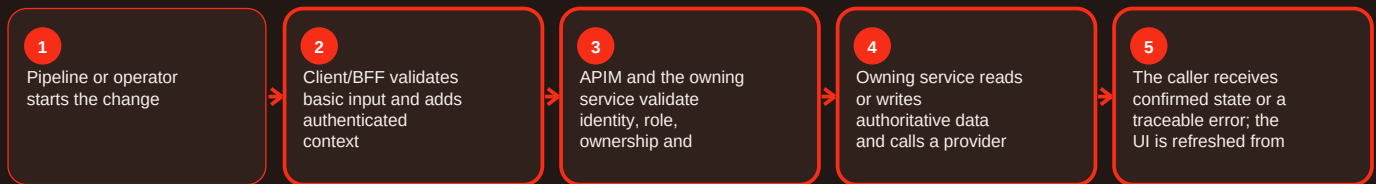
### Summary

Starter-secret prohibition is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For starter-secret prohibition, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed.

Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Provider credentials - Overview and responsibility

## Current implementation

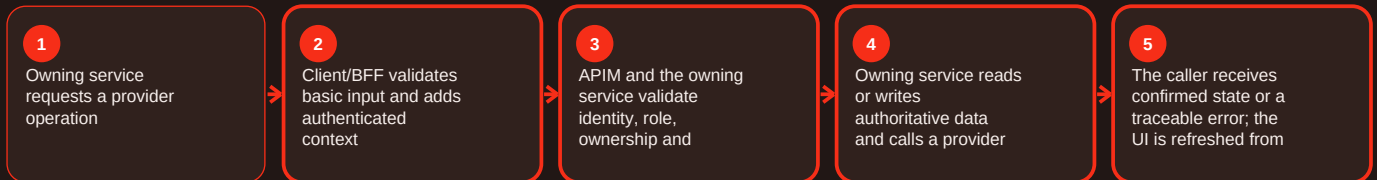
### Summary

Provider credentials is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For provider credentials, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/\*\*/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.



# Provider credentials - Functional behavior and data

## Current implementation

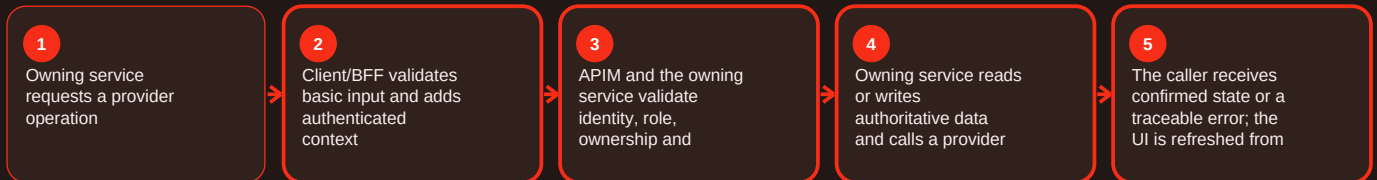
### Summary

Provider credentials is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For provider credentials, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/\*\*/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Provider credentials - Operations, errors and deployment

## Current implementation

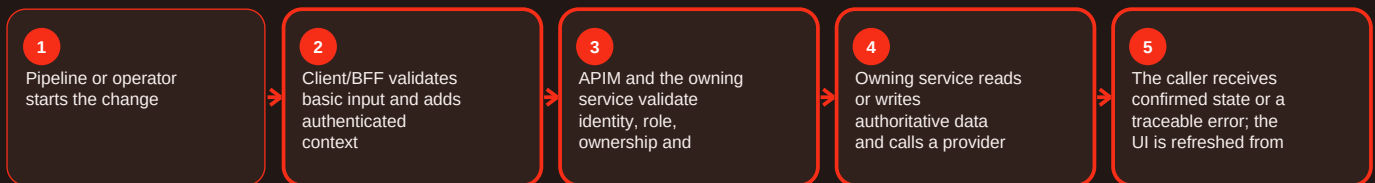
### Summary

Provider credentials is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For provider credentials, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Webhook hmac - Overview and responsibility

## Current implementation

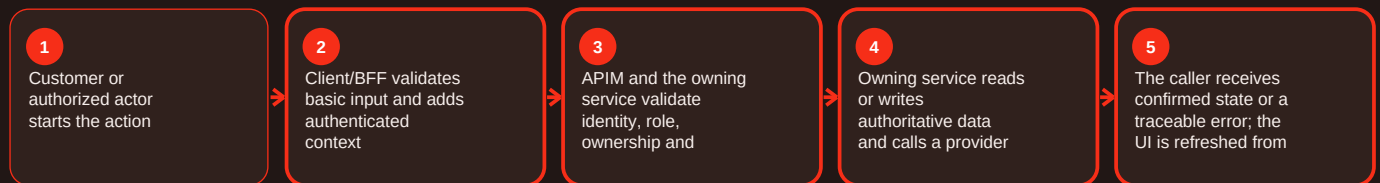
### Summary

Webhook hmac is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For webhook HMAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Webhook hmac - Functional behavior and data

## Current implementation

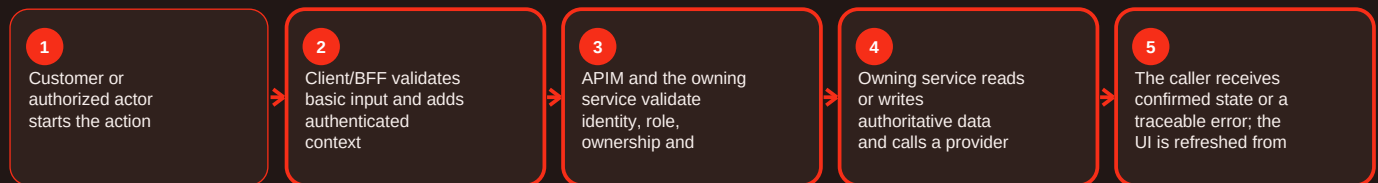
### Summary

Webhook hmac is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For webhook HMAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Webhook hmac - Operations, errors and deployment

## Current implementation

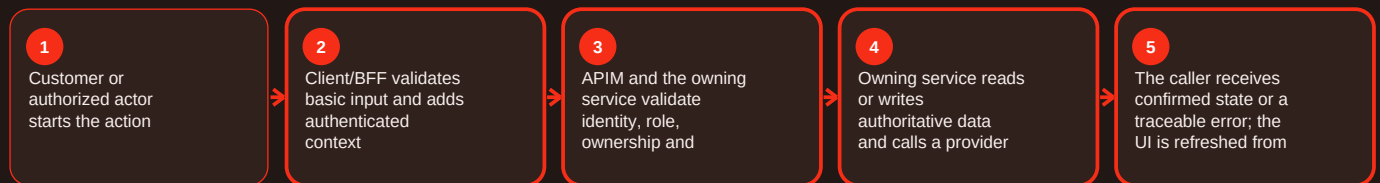
### Summary

Webhook hmac is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For webhook HMAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Idempotency / replay - Overview and responsibility

## Current implementation

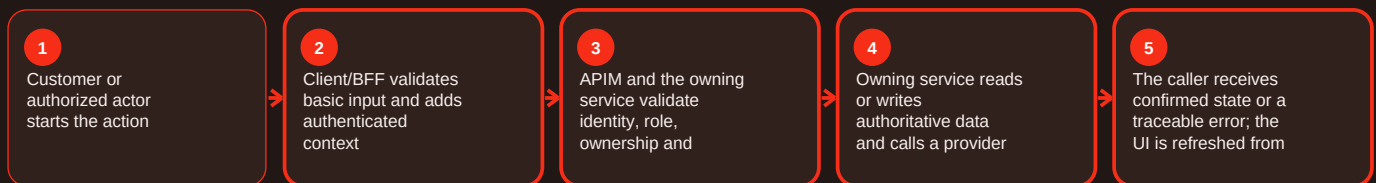
### Summary

Idempotency / replay is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For idempotency/replay, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Idempotency / replay - Functional behavior and data

## Current implementation

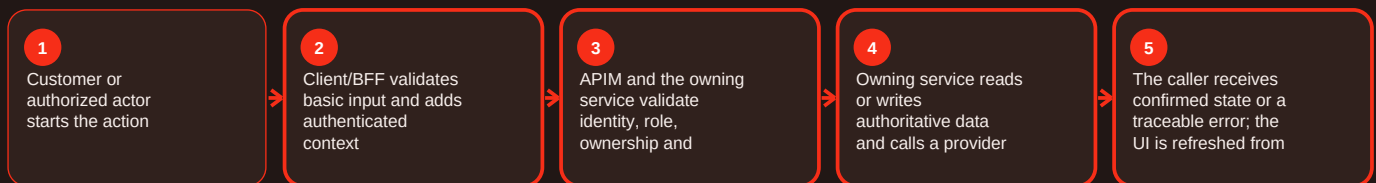
### Summary

Idempotency / replay is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For idempotency/replay, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Idempotency / replay - Operations, errors and deployment

## Current implementation

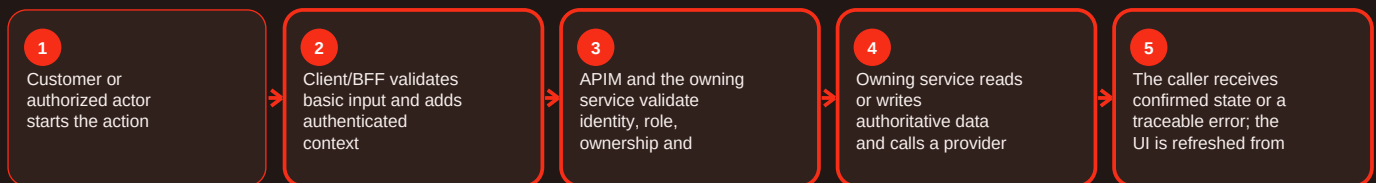
### Summary

Idempotency / replay is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For idempotency/replay, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user's order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.



# Input validation - Overview and responsibility

## Current implementation

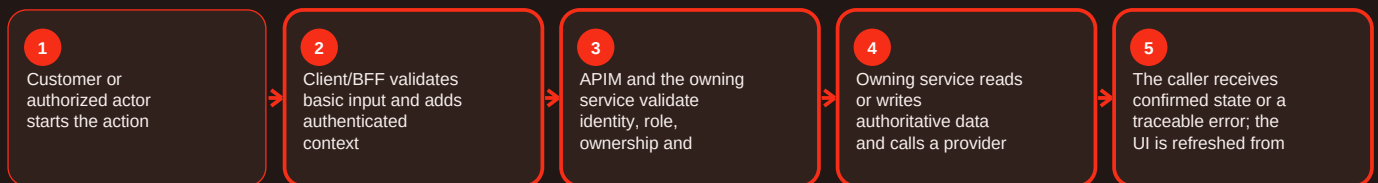
### Summary

Input validation is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For input validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Input validation - Functional behavior and data

## Current implementation

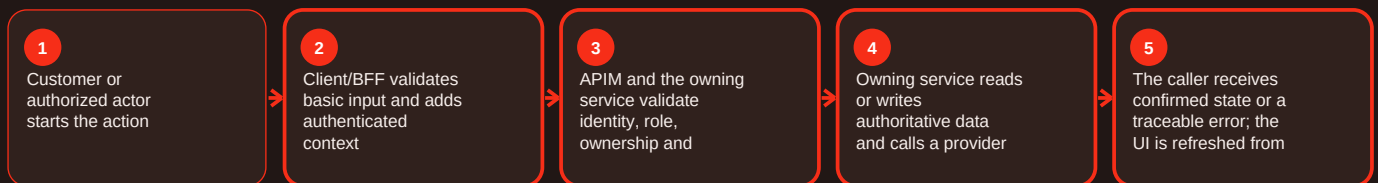
### Summary

Input validation is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For input validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Input validation - Operations, errors and deployment

## Current implementation

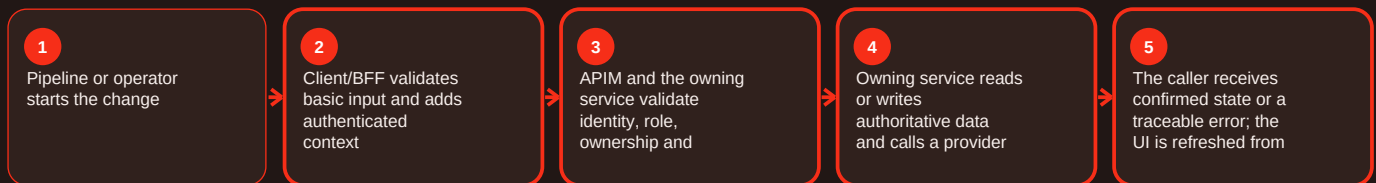
### Summary

Input validation is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For input validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Pii logging - Overview and responsibility

## Current implementation

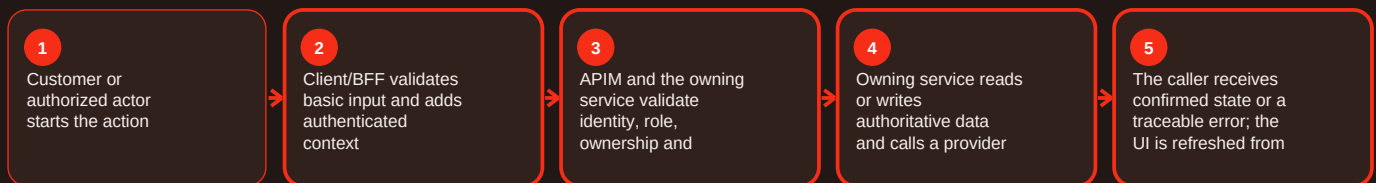
### Summary

Pii logging is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For PII logging, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Pii logging - Functional behavior and data

## Current implementation

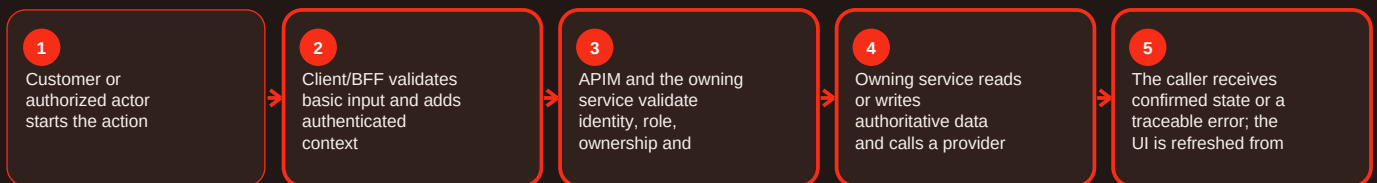
### Summary

Pii logging is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For PII logging, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Pii logging - Operations, errors and deployment

## Current implementation

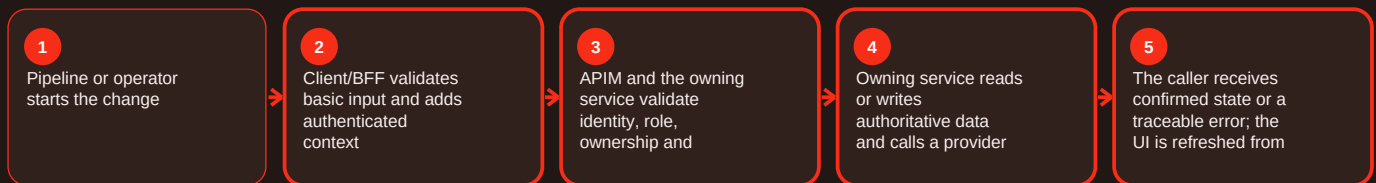
### Summary

Pii logging is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Pii logging, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Consent - Overview and responsibility

## Current implementation

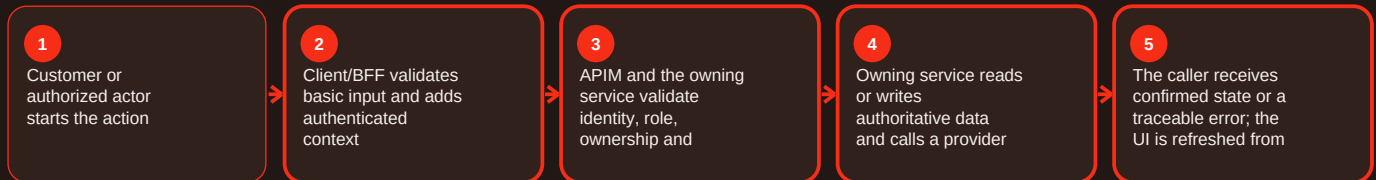
### Summary

Consent is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

### Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

### Process flow



### Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

### Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

# Operational checklist and documented gaps

## Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.
- Confirm least privilege and audit evidence for the sensitive action.

## Repository gaps that must stay visible

- APIM README cites `infra/apim/craves-openapi.yaml`, but that literal artifact was not resolvable on main during this evidence snapshot.
- `customer-web-next` README names `azure-pipelines-customer-web-next.yml`, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

## Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.